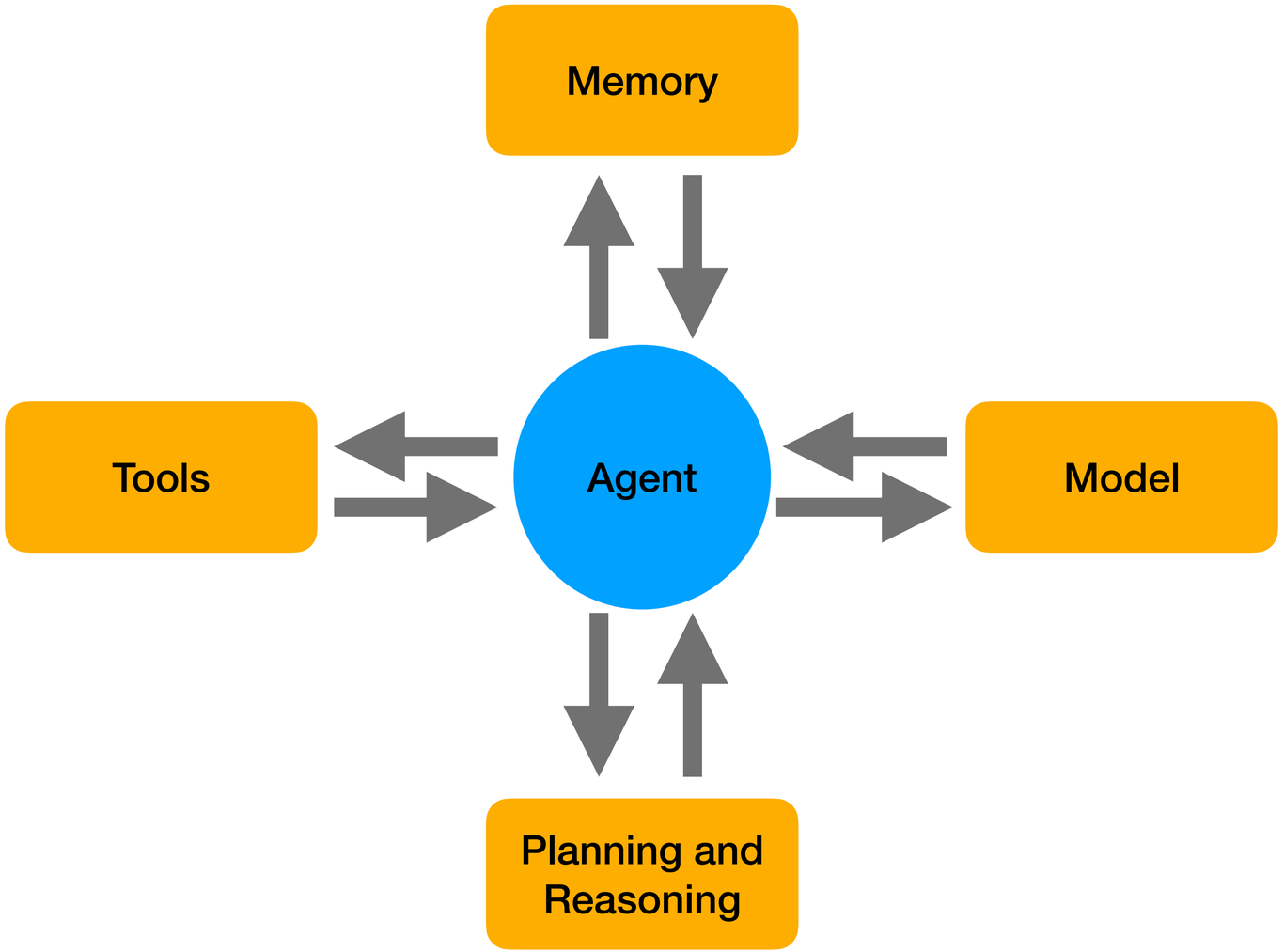
Tool Engineering and Model Context Protocol

Making Agents See And Do

Abhijith Neerkaje & Ajay Shenoy

Components of an agentic systems



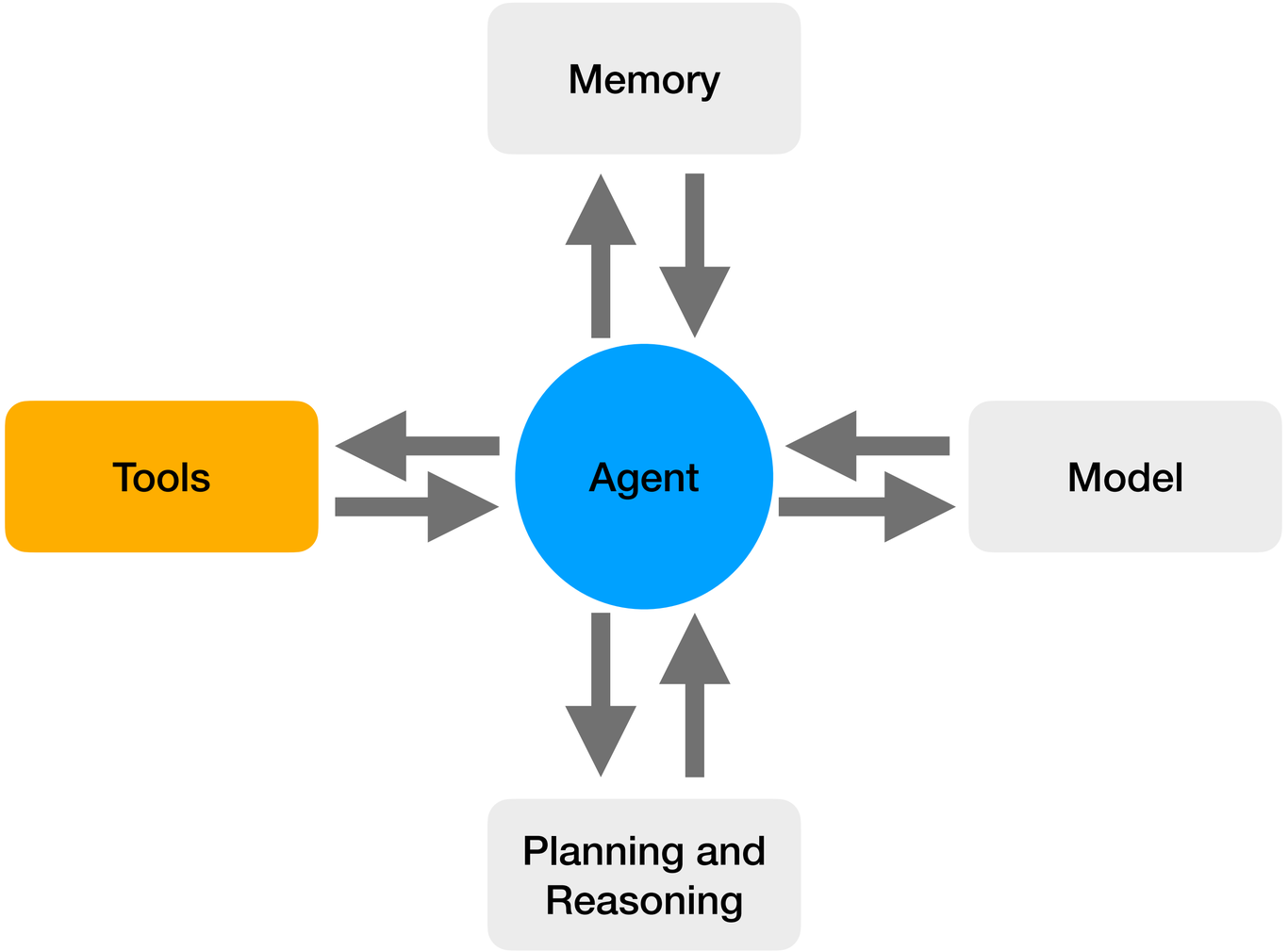
The fundamental problem

LLMs are brilliant but limited

- Language models excel at reasoning and text generation
- But they can't interact with the real world
- No web access, no code execution, no API calls, no file system

Solution: Give LLM Tools

Let us now focus on the tools



What is a tool?

Definition: Tools are external functions, APIs, or systems that an AI agent can invoke to gather information, perform actions, or interact with the environment.

Knowledge & Data Tools

- Databases
- Vector Stores

Computation Tools

- Optimizers
- Calculators

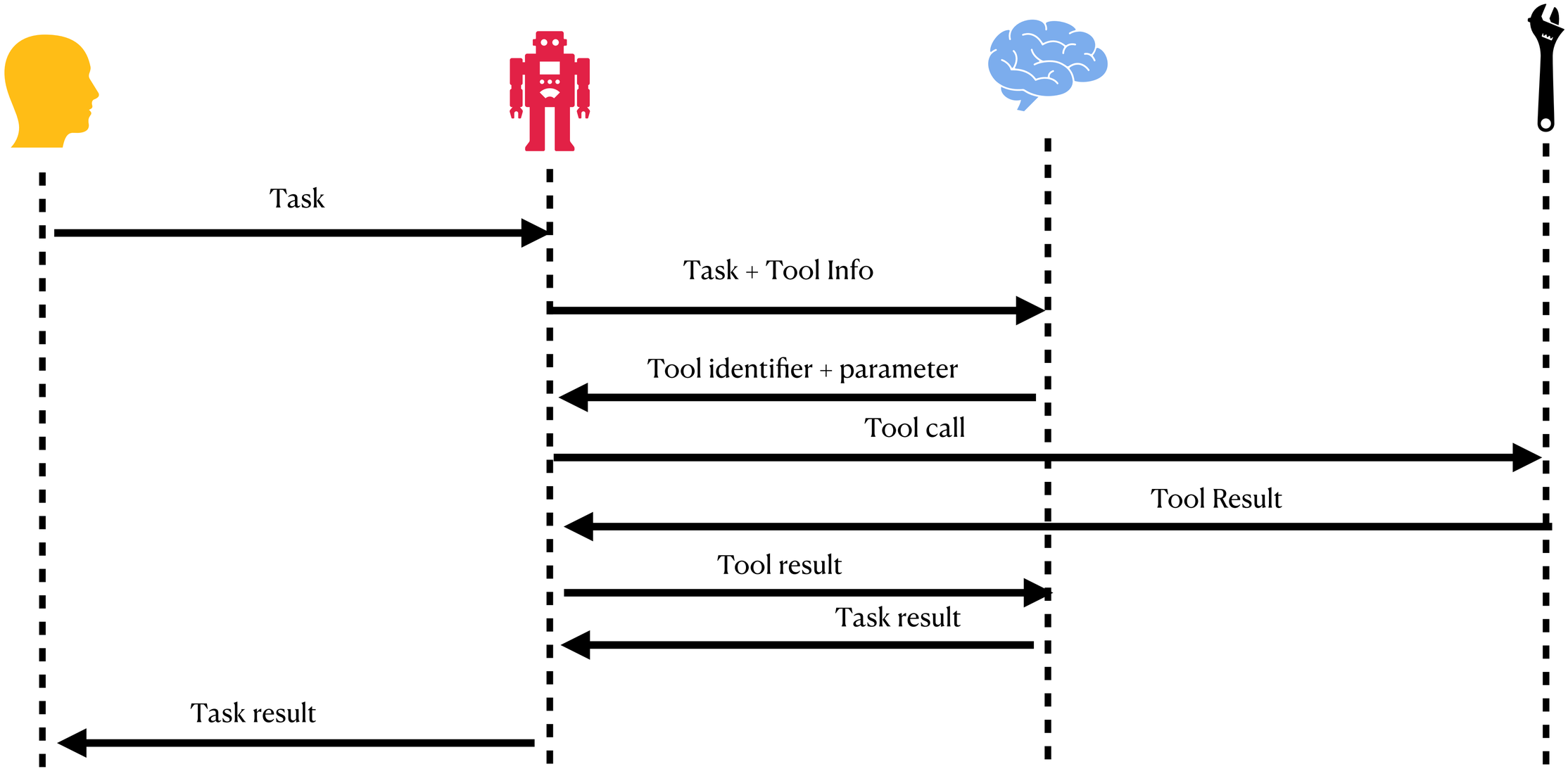
Communication Tools

- Email service
- Slack / WhatsApp or CRM connectors

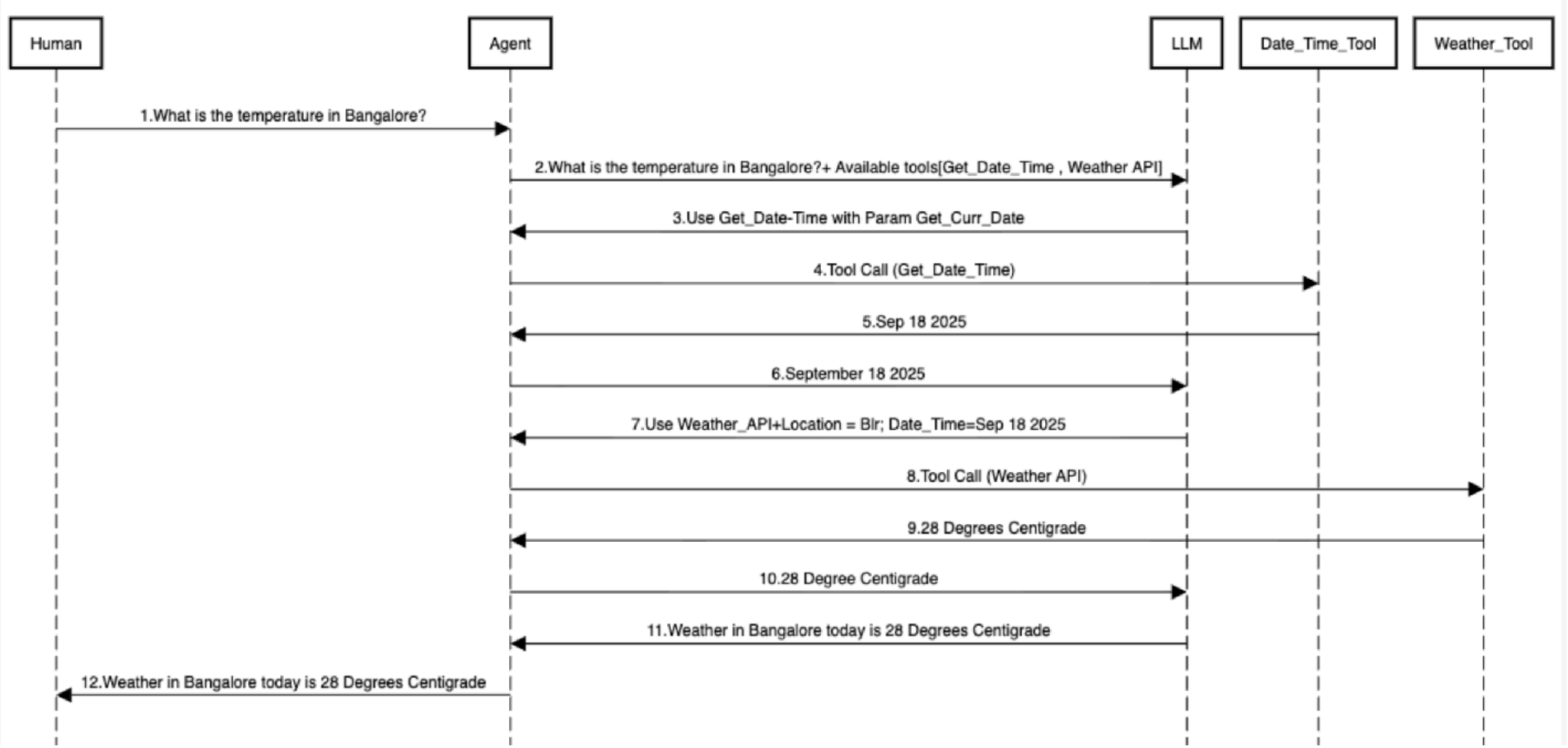
External Service APIs

- Payment Gateway
- Weather API

How do agents utilize tools?



Why is using using tools challenging?



- LLMs can hallucinate while choosing the tool
- LLMs can hallucinate while providing parameters
- LLMs hallucinate while interpreting results

Designing Tools

- Tools need to be carefully designed to ensure AI agents are:
  - Reliable
  - Efficient
  - Cost Effective

Tools Engineering

Tools Engineering is the discipline of designing interfaces between AI agents and external systems to ensure agents are reliable, autonomous, and effective.

Tool Definition

How tools are described to the AI in a structured, interpretable format that enables proper understanding and execution.

Execution Environment

Where and how tools run safely, including sandboxing, resource management, and security considerations.

Result Handling

Processing and interpreting tool outputs to provide meaningful feedback and enable decision-making.

Error Management

Implementing graceful failure modes and recovery mechanisms to maintain system reliability when tools fail or produce unexpected results.

Composition

Chaining tools together for complex workflows, enabling sophisticated multi-step processes and conditional logic flows.

Let us build a simple tool

- **Task:** Take a number from the user and return it's square root

```python
# --- Tool ---
def square_root(number: float) -> float:
    """Calculate the square root of a non-negative number."""
    if number < 0:
        raise ValueError("Cannot compute the square root of a negative number.")
    return sqrt(number)


square_root_tool = StructuredTool.from_function(
    func=square_root,
    name="square_root",
    description="Calculates the square root of a given non-negative number."
)
```

Error handling

Name of the tool

What the tool does?

Why don’t you build

Let us build a economic order quantity calculator

- Economic Order Quantity (EOQ) Formula:
- $EOQ = \sqrt{(2 \times D \times S / H)}$

Where:

D = Demand rate (units per time period, e.g., units per year)

S = Setup cost (cost per order, e.g., cost to place one order)

H = Holding cost (cost to hold one unit in inventory for one time period)

The EOQ formula helps determine the optimal order quantity that minimizes the total cost of inventory management by balancing ordering costs and holding costs.

Tools should act like building blocks not prewired circuits

Bad Design

```python
from langchain.tools import tool

@tool
def inventory_manager(task: str):
    """Do forecasts, set inventory policy, or send reorder request."""
    if task == "forecast": return "📈 Forecast done"
    if task == "policy": return "📦 Policy computed"
    if task == "reorder": return "✅ Reorder placed"
```

Good Design

```python
@tool
def compute_forecast(data: str): return "📈 Forecast done"

@tool
def compute_inventory_policy(data: str): return "📦 Policy computed"

@tool
def send_reorder_request(item: str): return "✅ Reorder placed"
```

Allowing LLMs to choose from a array of tools, helps build a flexible system that can evolve with changing needs of the goal and environment.

Tools should be designed for AI consumption

Bad Design

```python
#  Bad: Vague + LLM-unfriendly
@tool
def check_stock(product):
    """Check stock."""
    # returns raw DB object, unclear to LLM
    return {"qty": 12, "loc": "W1"}
```

Good Design

```python
# Good: Clear + LLM-friendly
@tool
def check_stock(product_id: str) -> str:
    """
    Check available stock for a given product.
    Input: product_id (string, e.g., "SKU123")
    Output: JSON with fields {available_qty:int, location:str}
    Errors: "ProductNotFound" if SKU is invalid
    """
    return '{"available_qty": 12, "location": "Warehouse W1"}'
```

Bad Design

```
{
  "error": "ERR42",
  "message": "Invalid input format",
  "code": 400
}
```

Good Design

```
{
  "error": "Invalid date format",
  "message": """The 'start_time' must be in ISO 8601 format (e.g., 2025-09-17T15:00:00Z). "
  "Please correct and try again."""
}
```

Tool description should have information about usage inputs outputs and errors.
Tool output should be optimized for LLM use.

Tools should be predictable and specific

Bad Design

```python
#  Bad: One tool, LLM must decide logic internally
@tool
def enrich_catalog(product: dict, task: str) -> dict:
    """Do catalog enrichment (improve title, improve description, or add attributes).
    Input: product dict + task type.
    Output varies depending on task."""
    if task == "title":
        return {"improved_title": improve_title(product["title"])}
    elif task == "description":
        return {"improved_description": improve_description(product["description"])}
    elif task == "attributes":
        return {"completed_attributes": infer_attributes(product["attributes"])}
    else:
        return {"error": "Unknown task"}
```

Good Design

```python
# ✅ Good: Separate, deterministic tools
@tool
def improve_title(product: dict) -> dict:
    """Enhance product title for clarity & SEO.
    Input: {title}, Output: {improved_title}"""
    return {"improved_title": run_title_model(product["title"])}

@tool
def improve_description(product: dict) -> dict:
    """Generate better product description.
    Input: {description}, Output: {improved_description}"""
    return {"improved_description": run_description_model(product["description"])}

@tool
def add_missing_attributes(product: dict) -> dict:
    """Fill missing catalog attributes (color, material, size).
    Input: {attributes}, Output: {completed_attributes}"""
    return {"completed_attributes": infer_attributes(product["attributes"])}
```

LLMs can easily determine what each tool is doing. It need not make a decision of **how** to use the tool. Tools should help make the system more deterministic not probabilistic!

Tools should be safe

```python
# Good: Safe destructive tool design
@tool
def delete_product(product_id: str, user_id: str) -> str:
    """
    Delete product from catalog.
    Requires: product_id, authenticated user_id.
    Behavior: logs action, alerts admin, allows rollback.
    """
    if not is_authorized(user_id, "delete_product"):
        return "Unauthorized"

    backup = db.get(product_id)              # audit + rollback
    db.delete(product_id)
    log_action("delete", product_id, user_id)
    alert_admin(f"Product {product_id} deleted by {user_id}")

    return f"Deleted {product_id}. Rollback ID: {save_backup(backup)}"
```

- All tool access should be authenticated authorized and audited.
- Destructive tools (example: deleting data in database) should be capable of reversing the effects and alert humans.

Activity: Refactor a tool specification

You are designing a tool for scheduling meetings in an enterprise AI assistant. The current tool spec was written quickly by a developer, but it violates multiple principles.Your task: Refactor the tool spec to follow best practices.

Bad Tool Spec

```
{
  "name": "do_meeting",
  "description": "Use this tool to arrange everything about meetings.",
  "parameters": {
    "info": {
      "type": "string",
      "description": "Meeting details, can be date, time, participants, purpose, or anything else."
    }
  }
}
```

- **Best Practices**
- Building blocks, not prewired circuits
- Designed for AI consumption
- Predictable & specific
- Safe

Tool Execution Environments Best Practices

Environment	Principle	Example
Sandboxed	Docker containers with limited resources	Price-optimization tool runs in a container capped at 1 CPU / 512MB RAM
	Network restrictions & timeouts	Stock-check tool only allowed to call warehouse API, with 5s timeout
	File system isolation	Log-writing tool can write only to `/sandbox/logs/`, not system files
Managed Services	Serverless functions for stateless operations	Catalog-enrichment tool runs as a Lambda, scales automatically
	Managed databases for persistent storage	Forecasting tool stores results in BigQuery, not local disk
	API gateways for external integrations	Payment-tool calls external API only via API Gateway enforcing auth
Security Layers	Input validation & sanitization	Rejects malicious inputs like `product_id="DROP TABLE"`
	Output filtering & size limits	Product search tool capped to return 50 results max

Implementing Tools

Tool implementation requirements

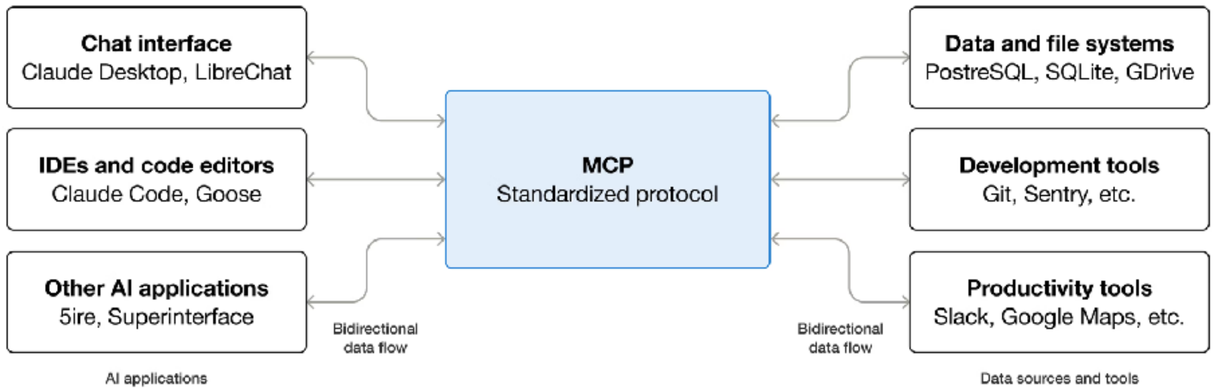
- **Functionality:** My agent should be able to interact with my tool and get the task done - duhh!!!
- **Interoperability:** My agent should be able to interact with tools built by others
- **Reuse:** I might want to expose my tools to other agents to use
- **Extensibility:** Can others extend my agent with their own tools?
- **Discoverability:** How can agents discover tools?

Model context protocol allows interoperability between Agents and Tools

What is MCP?

USB -C port for AI Applications

- MCP (Model Context Protocol) is an open-source standard from Anthropic for connecting AI applications to external systems.
- Adopted by major players like Google, Open AI, Microsoft etc
- **Promise:**Build once and use anywhere

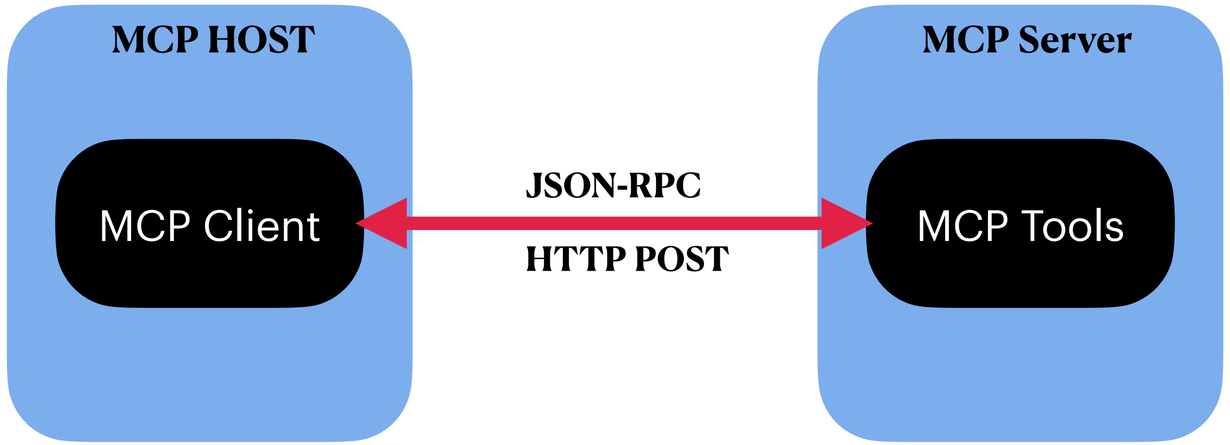


MCP is to AI assistants what HTTP was to the web — a common protocol so models, apps, and tools can “talk” to each other in a safe, standardized way.

Architecture of MCP

USB -C port for AI Applications

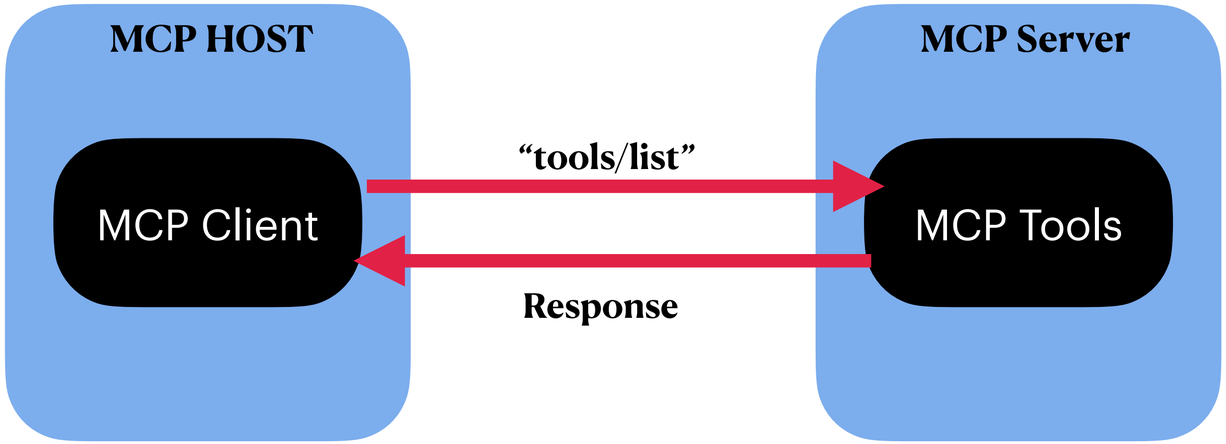
- MCP Host - Agent
- MCP Host has a MCP client per tool
- Clients invoke the tools
- Tools like Google calendar have a MCP server
- Servers define input output schema with field descriptions and tool descriptions
- Servers give tool behavior hints



Tool Lifecycle for MCP

2 Step Process

- Step 1 : Discover tools



Client request

```
{
  "id": "1",
  "jsonrpc": "2.0",
  "method": "tools/list",
  "params": {}
}
```

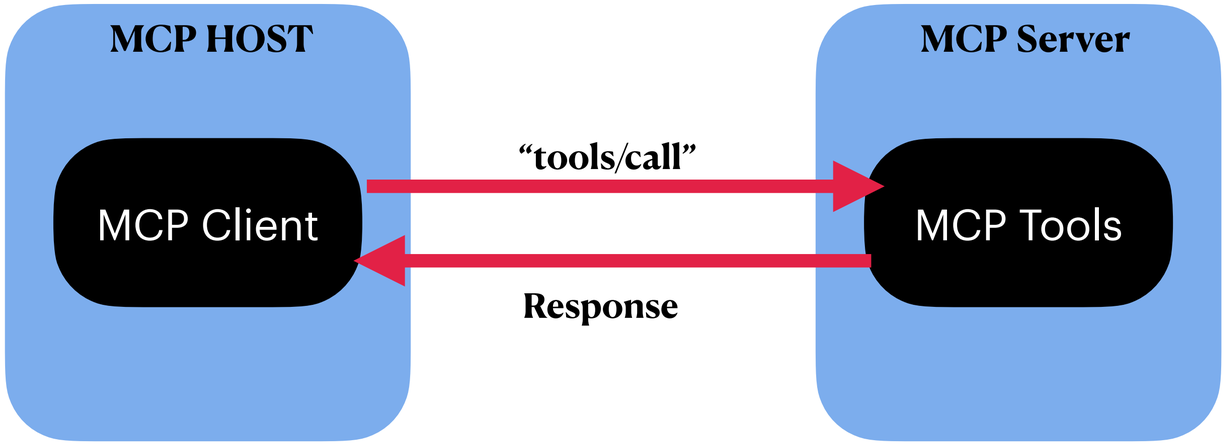
Server response

```
{
  "id": "1",
  "jsonrpc": "2.0",
  "result": {
    "tools": [
      {
        "name": "square_root",
        "description": "Calculates the square root of a given non-negative number.",
        "input_schema": {
          "type": "object",
          "properties": {
            "number": {
              "type": "number",
              "description": "A non-negative number"
            }
          },
          "required": ["number"]
        }
      },
```

Tool Lifecycle for MCP

2 Step Process

- Step 2 : MCP Tool Invocation



Client request

```
{
  "id": "2",
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "square_root",
    "arguments": {
      "number": 81
    }
  }
}
```

Server response

```
{
  "id": "2",
  "jsonrpc": "2.0",
  "result": {
    "content": [
      {
        "type": "text",
        "text": "9.0"
      }
    ],
    "isError": false
  }
}
```

Demo of MCP based tool implementation

- Tool Integration via MCP – Connects local and remote tools (e.g., time and weather) to the AI assistant.
- Dynamic Function Calling – LLM selects and invokes the right tool automatically based on user query.
- Local Tool Example – Retrieves current system time with timezone awareness.
- Remote Tool Example – Fetches live weather data from an external API using city name input.
- End-to-End Demo Flow – User asks → Assistant triggers tools → Results returned → Final AI-generated response.

It’s a wrap